

Sprint 2

Nivel 1

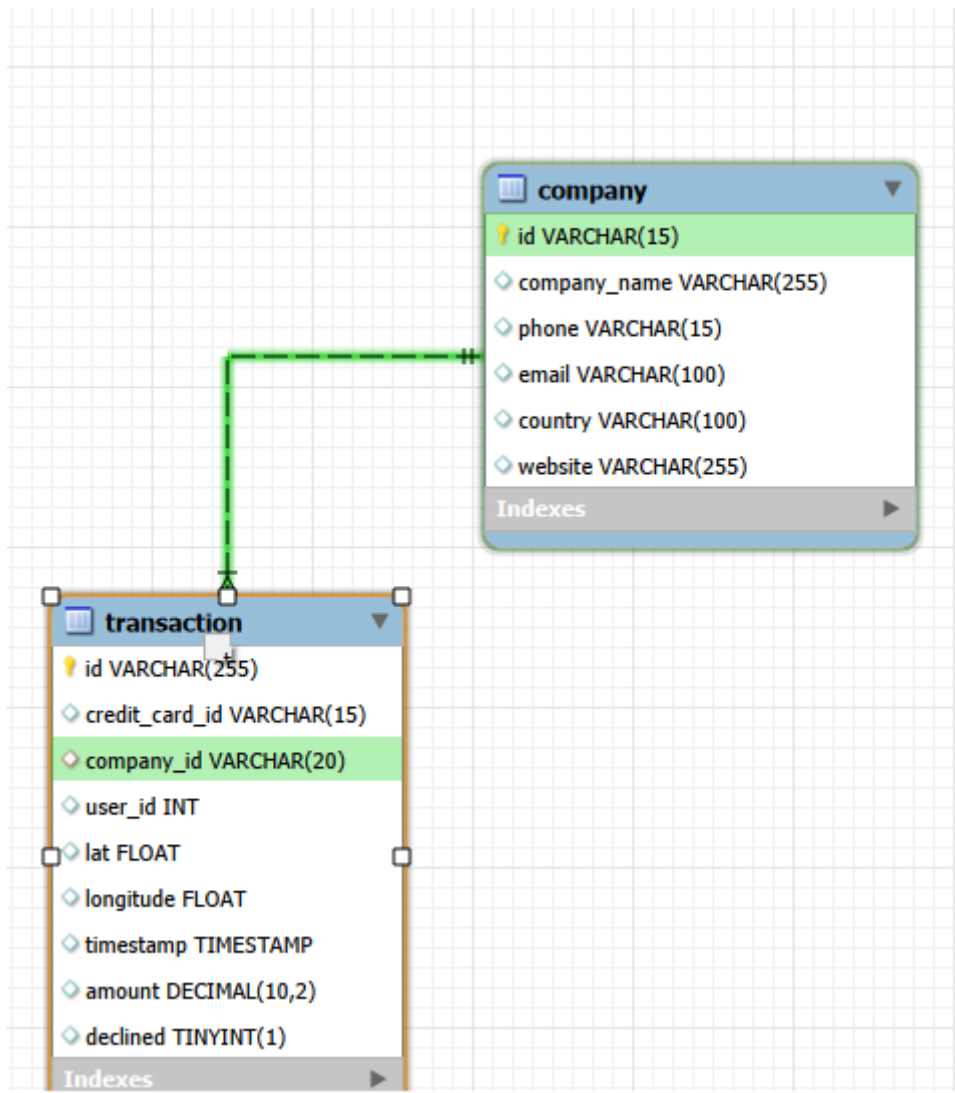
Ejercicio 1

A partir de los documentos adjuntos (estructura_datos y datos_introducir), importa las dos tablas. Muestra las principales características del esquema creado y explica las diferentes tablas y variables que existen. Asegúrate de incluir un diagrama que ilustre la relación entre las distintas tablas y variables.

Respuesta: La base de datos transactions se compone de dos tablas principales: company y transaction. Estas tablas están diseñadas para almacenar información sobre empresas y las transacciones que se realizan en ellas.

En la tabla company se almacena la información de la empresa s, como el nombre de la empresa, número de teléfono, dirección, país y página web, su Primary Key es la columna de ID

En la tabla transaction se almacena la información de las transacciones como su ubicación, fecha de la transacción, el estado y el monto, se hace referencia mediante una Foreign Key a la tabla company, en una relación de muchos a uno



Ejercicio 2

Utilizando JOIN realizarás las siguientes consultas:

Listado de los países que están generando ventas.

Desde cuántos países se generan las ventas.

Identifica a la compañía con la mayor media de ventas.

Respuesta:

Listado de países donde se generan ventas:

SQL File 19*

```

2 SELECT *
3 FROM transaction;
4
5 #Listado de los países que están generando ventas.
6 -- 1. Listado de los países que están generando ventas
7 -- Se seleccionan los países únicos de la tabla company que tienen transacciones no rechazadas.
8 SELECT DISTINCT c.country
9 FROM company c
10 JOIN transaction t ON c.id = t.company_id
11 WHERE t.declined = 0
12 ORDER BY c.country;

```

Result Grid

country
Australia
Belgium
Canada
China
France
Germany
Ireland

Result 3 x

Output

#	Time	Action	Message	Duration / Fetch
116578	11:24:00	SELECT * FROM transaction	100000 row(s) returned	0.109 sec / 0.578 sec
116579	11:51:51	INSERT INTO credit_card (d.iban, pan, pin, cvv, expiring_date) VALUES (CoU-293...	Error Code: 1146. Table 'transactions.credit_card' doesn't exist	0.125 sec
116580	12:12:39	SELECT DISTINCT c.country FROM company c JOIN transaction t ON c.id...	15 row(s) returned	0.500 sec / 0.000 sec

Query Completed

Desde cuántos países se generan las ventas:

Para efectos del ejercicio la cantidad de países desde donde se generan ventas la podríamos responder con la información en el módulo que aparece en el Action Output del ejercicio

Empresas con un amount mayor que la media de todas las transacciones

SQL File 19*

```

41 -- Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones.
42
43 SELECT DISTINCT company_name
44 FROM company
45 WHERE id IN (
46     SELECT company_id
47     FROM transaction
48     WHERE amount > (
49         SELECT AVG(amount)
50         FROM transaction
51     )
52 );

```

Result Grid

company_name
Ac Fermentum Incorporated
Magna A Nique Industries
Fusce Corp.
Convalis In Incorporated
Ante Iaculis Nec Foundation
Donec Ltd
Sed Nunc Ltd

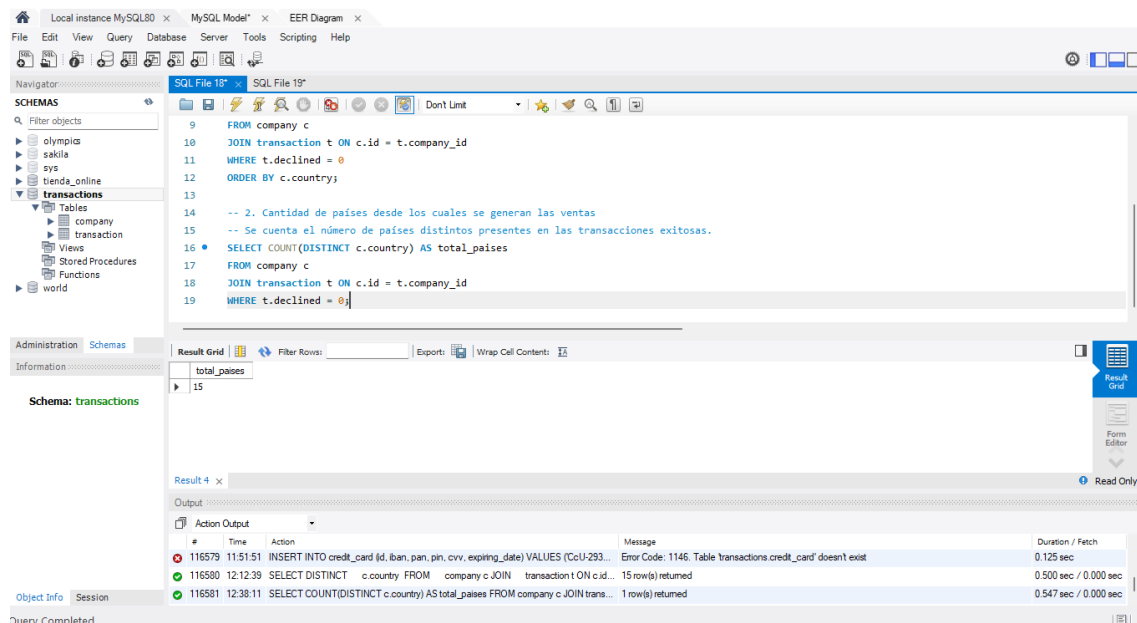
company 28 x

Output

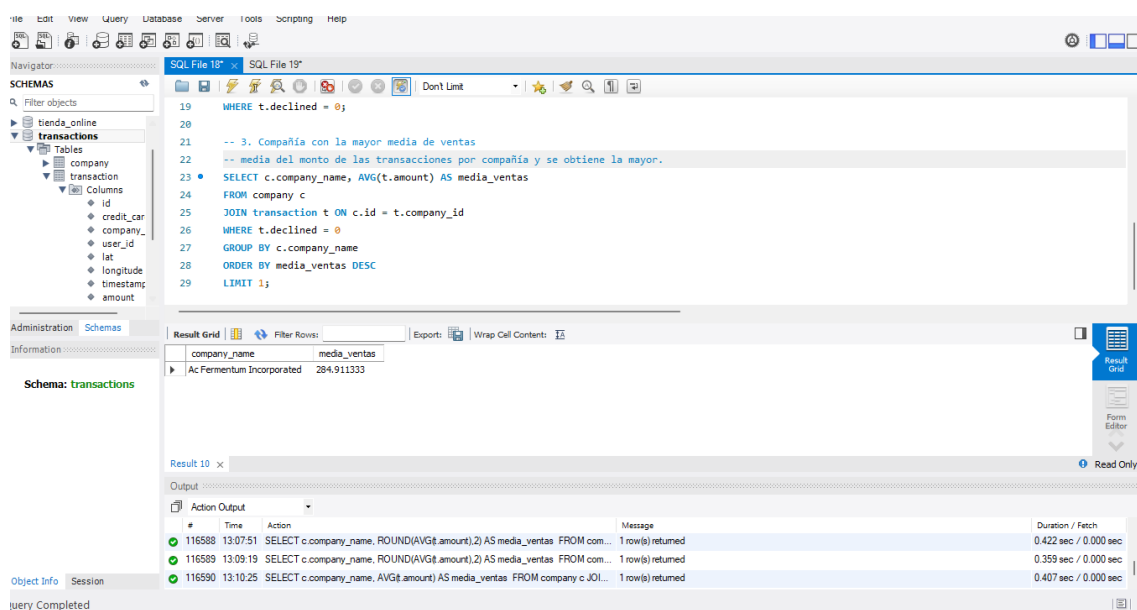
#	Time	Action	Message	Duration / Fetch
116613	14:12:27	SELECT *, (SELECT AVG(amount) FROM transaction) AS media_total FROM transaction	100000 row(s) returned	0.047 sec / 0.203 sec
116614	14:13:39	SELECT DISTINCT company_name FROM company WHERE id IN (SELECT company_id FRO...	100 row(s) returned	0.125 sec / 0.000 sec

Query Completed

Empresas que carecen de transacciones o anterior, pero decidí mostrarla directamente creando una nueva columna mediante la función count y renombrándola total _países para ser más específico.



Identifica a la compañía con la mayor media de ventas:



Ejercicio 3

Utilizando sólo subconsultas (sin utilizar JOIN):

- Muestra todas las transacciones realizadas por empresas de Alemania.
- Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones.
- Eliminarán del sistema las empresas que carecen de transacciones registradas, entrega el listado de estas empresas.

Transacciones realizadas por empresas de Alemania

SQL File 19*

```

33 -- Muestra todas las transacciones realizadas por empresas de Alemania.
34 SELECT *
35 FROM transaction
36 WHERE company_id IN (
37     SELECT id
38     FROM company
39     WHERE country = 'Germany'
40 );
41 -- Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones.

```

Result Grid

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00138038-206D-4C03-94B7-63A2676EB984	CcS-4899	b-2222	318	41.3781	12.447	2020-03-25 10:43:43	426.36	0
0013C186-3684-4D6C-8154-E2B3FEB0A8E9	CcS-5070	b-2222	489	41.3814	2.18176	2020-12-17 18:15:37	316.90	0
00201A11-2E62-44C4-941D-199FC80B77F0	CcU-3512	b-2222	193	55.5704	-3.65129	2021-01-22 23:44:27	453.04	0
00235618-0A5C-4D49-ADCB-83A940D8923	CcS-8137	b-2222	3556	59.8421	18.729	2020-09-09 15:43:19	263.14	0
005A5A7B-3F1A-4B6C-9B15-1625A78C9C38	CcS-8998	b-2222	4417	41.1591	-8.63905	2024-05-15 09:10:11	442.01	0

transaction 21 x

Output

#	Time	Action	Message	Duration / Fetch
116597	13:47:37	SELECT * FROM company_name	Error Code: 1146. Table 'transactions.company_name' doesn't exist	0.047 sec
116598	13:48:01	SELECT company_name FROM company	100 row(s) returned	0.000 sec / 0.000 sec
116599	13:50:49	SELECT distinct id FROM transaction	100000 row(s) returned	0.015 sec / 0.391 sec
116600	13:51:19	SELECT distinct company_id FROM transaction	100 row(s) returned	0.016 sec / 0.000 sec
116601	13:51:46	SELECT * FROM transaction WHERE company_id IN (SELECT id FROM company WHERE	13291 row(s) returned	0.032 sec / 0.109 sec
116602	13:52:06	SELECT * FROM transaction WHERE company_id IN (SELECT id FROM company WHERE	13291 row(s) returned	0.000 sec / 0.094 sec

Query Completed

Lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones.

Navigator

Schemas

- e_commerce_analytics
 - Tables
 - dim_companies
 - dim_credit_cards
 - dim_users
 - fact_transactions
 - Views
 - Stored Procedures
 - Functions
- olympics
- sakila
- sys
- tienda_online
- transactions
 - Administration
 - Schemas

Schema: transactions

SQL File 4*

```

1 SELECT DISTINCT company_name
2 FROM company
3 WHERE id IN (SELECT company_id
4             FROM transaction
5             WHERE amount > (SELECT AVG(amount)
6                             FROM transaction
7                             ))
8 );

```

Result Grid

company_name
Ac Fermentum Incorporated
Magna A Neque Industries
Fusce Corp.
Convallis In Incorporated
Ante Iaculis Nec Foundation
Donec Ltd

company 1 x

Output

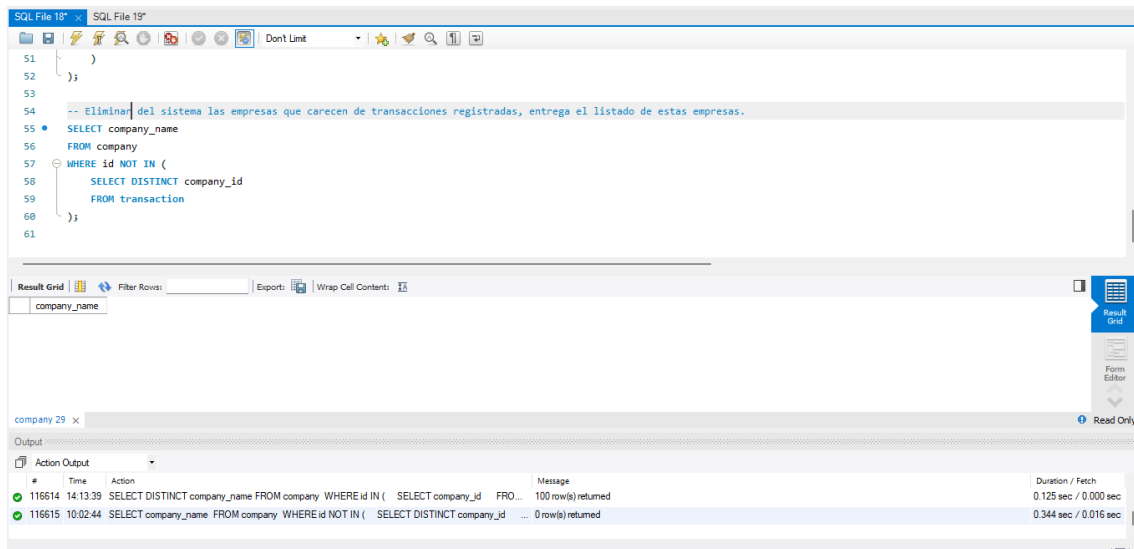
#	Time	Action	Message	Duration / Fetch
21	11:08:35	INSERT IGNORE INTO dim_credit_cards (card_id, iban, card_type, expiration_date, c...	Error Code: 1054. Unknown column 'expiration_date' in field list	0.000 sec
22	11:09:40	INSERT IGNORE INTO dim_credit_cards (card_id, iban) VALUES ('Cc-4458', 'ES0000...	1 row(s) affected, 1 warning(s): 1364 Field 'user_id' doesn't have a default value	0.000 sec
23	11:10:41	INSERT IGNORE INTO dim_credit_cards (card_id, iban, user_id) VALUES ('Cc-4458', '...	0 row(s) affected, 1 warning(s): 1062 Duplicate entry 'Cc-4458' for key 'dim_credit_cards...	0.000 sec
24	11:11:58	INSERT INTO fact_transactions (transaction_id, card_id, company_id, timestamp, a...	1 row(s) affected	0.000 sec
25	11:12:02	SELECT * FROM fact_transactions WHERE transaction_id = '000447FE-B650-4DCF-8...	1 row(s) returned	0.000 sec / 0.000 sec
26	11:12:16	SELECT transaction_id, amount, timestamp FROM fact_transactions WHERE transac...	1 row(s) returned	0.015 sec / 0.000 sec
27	11:14:45	DELETE FROM fact_transactions WHERE transaction_id = '000447FE-B650-4DCF-85...	1 row(s) affected	0.000 sec

Object Info Session

Query Completed

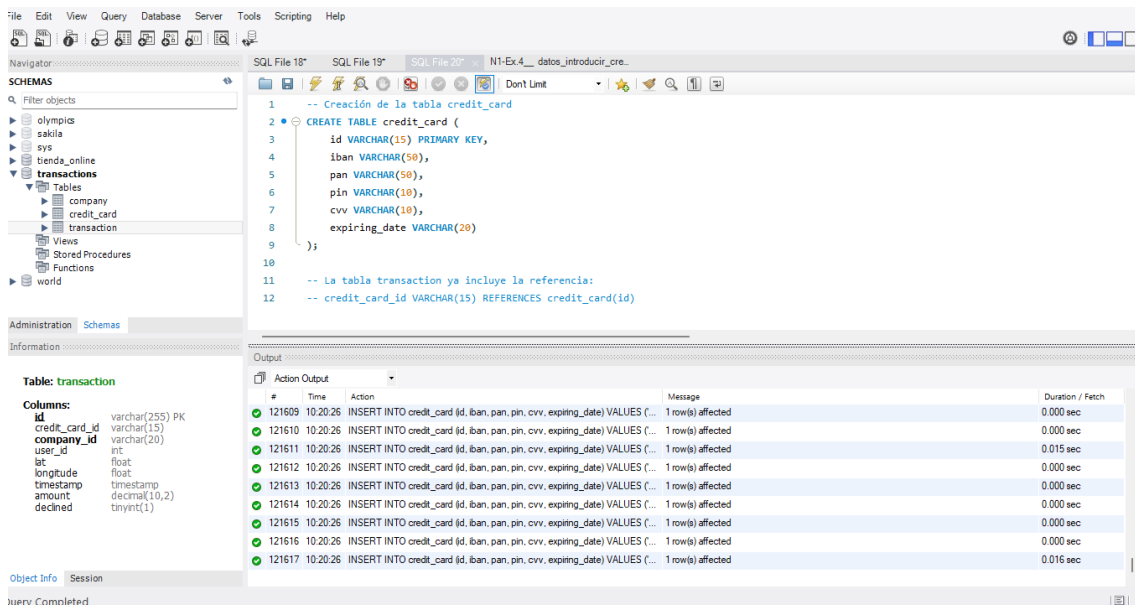
Eliminarán del sistema las empresas que carecen de transacciones registradas, entrega el listado de estas empresas.

Respuesta: No se encuentran empresas que carezcan de transacciones

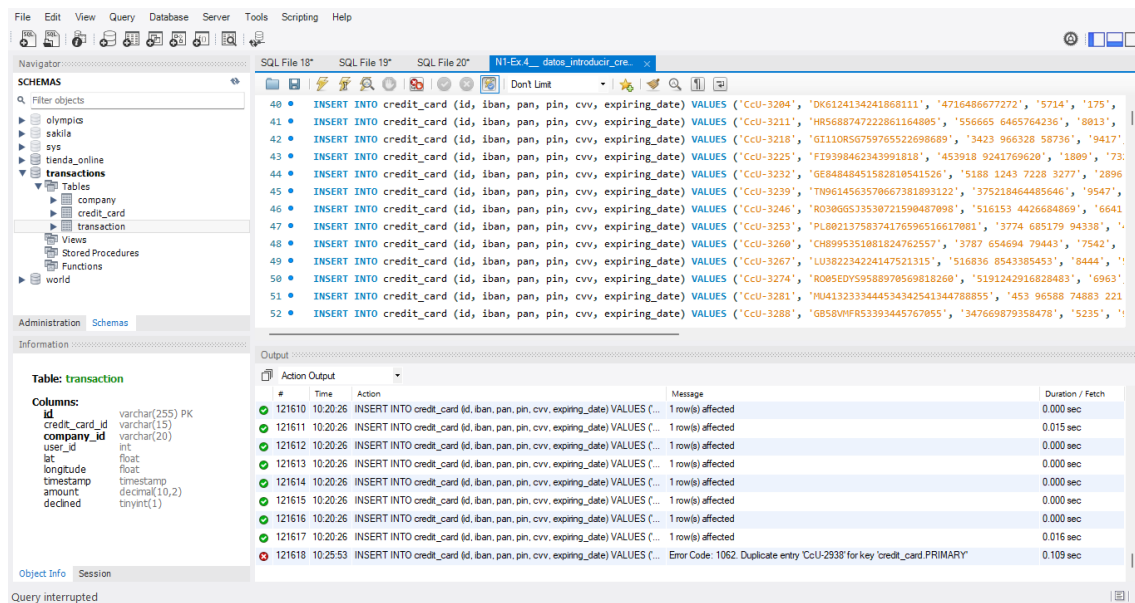


Ejercicio 4

Tu tarea es diseñar y crear una tabla llamada "credit_card" que almacene detalles cruciales sobre las tarjetas de crédito. La nueva tabla debe ser capaz de identificar de forma única cada tarjeta y establecer una relación adecuada con las otras dos tablas ("transaction" y "company"). Después de crear la tabla será necesario que ingreses la información del documento denominado "datos_introducir_credit". Recuerda mostrar el diagrama y realizar una breve descripción del mismo.



Ingreso información tabla credit_card



Tambien se realiza la relacion entre las tablas

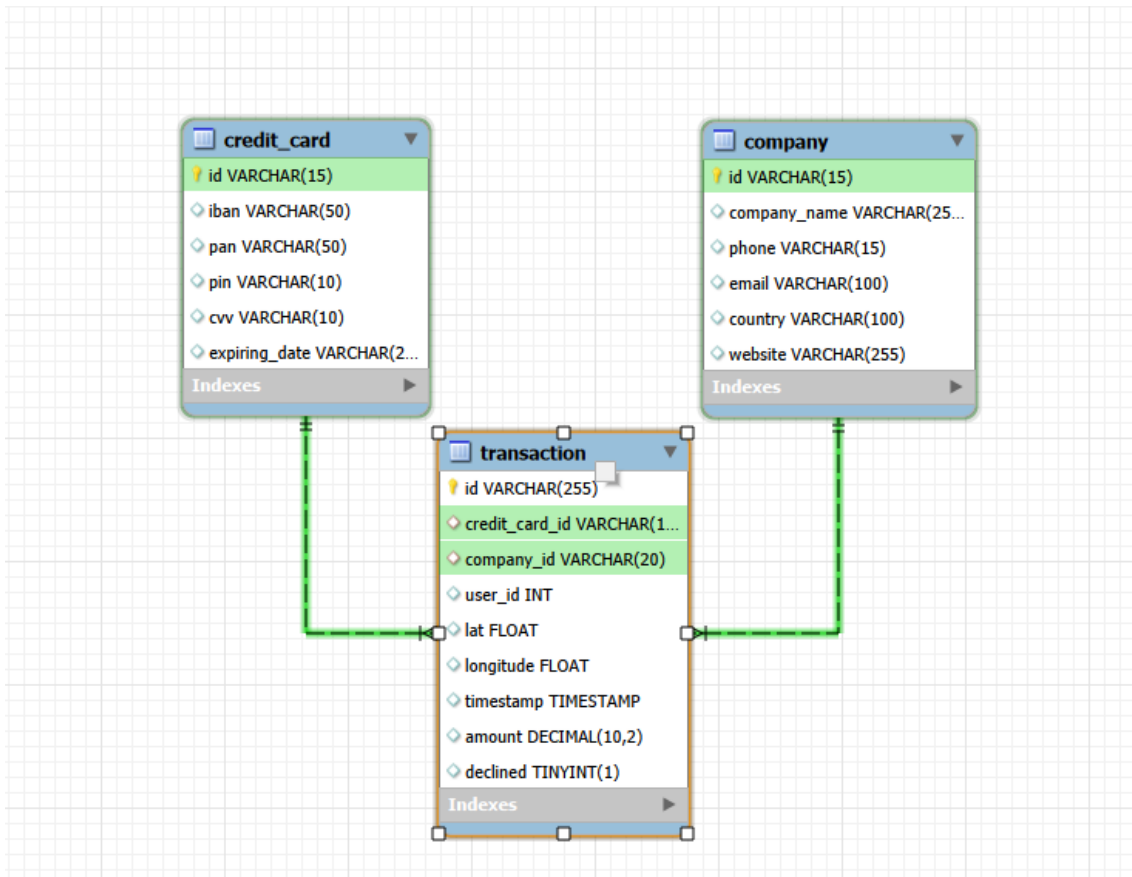
```

1      -- 1. Relación entre Transacciones y Tarjetas de Crédito
2      • ALTER TABLE fact_transactions
3        ADD CONSTRAINT FK_transactions_credit_card
4        FOREIGN KEY (card_id) REFERENCES credit_card (card_id)
5        ON DELETE CASCADE
6        ON UPDATE CASCADE;
7
8      -- 2. Relación entre Transacciones y Compañías
9      • ALTER TABLE fact_transactions
10     ADD CONSTRAINT FK_transactions_companies
11     FOREIGN KEY (company_id) REFERENCES dim_companies (company_id)
12     ON DELETE CASCADE
13     ON UPDATE CASCADE;

```

Nuevo diagrama

El diagrama presenta un modelo de estrella simplificado donde la tabla



TRANSACTION actúa como el núcleo central.

- Relación con COMPANY: Existe una relación de uno a muchos (1:N), donde una empresa puede estar vinculada a múltiples transacciones, pero cada transacción pertenece a una única empresa a través del campo company_id.
- Relación con CREDIT_CARD: Se ha establecido una relación de uno a muchos (1:N) entre las tarjetas y las transacciones. Una tarjeta de crédito puede ser utilizada para realizar múltiples compras, quedando registrada en la tabla de transacciones mediante el campo credit_card_id, que actúa como clave foránea.

Este diseño garantiza la integridad de los datos, permitiendo rastrear qué tarjeta se usó en cada venta y a qué empresa pertenece dicha operación.

Ejercicio 5

El departamento de Recursos Humanos ha identificado un error en el número de cuenta asociado a su tarjeta de crédito con ID CcU-2938. La información que debe mostrarse para este registro es: TR323456312213576817699999. Recuerda mostrar que el cambio se realizó.

Consulta antes del cambio del IBAN

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- company_name
- phone
- email
- country
- website
- Indexes
- Foreign Keys
- Triggers
- credit_card
 - Columns
 - id
 - iban
 - pan
 - pin
 - cvv

Administration Schemas

Information

Schema: transactions

SQL File 18" x SQL File 19"

58 SELECT DISTINCT company_id
59 FROM transaction
60 }
61 -- no hay empresas
62
63 #Ejercicio 5
64 -- cambio en el base de datos del numero iban para una tarjeta de credito
65 • SELECT *
66 FROM credit_card
67 WHERE id = "CcU-2938";

Result Grid

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	10/30/22

credit_card 34 x

Output

#	Time	Action	Message	Duration / Fetch
121621	11:00:21	SELECT * FROM credit_card WHERE id = CcU-2938	Error Code: 1054. Unknown column 'CcU' in 'where clause'	0.000 sec
121622	11:00:46	SELECT * FROM credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.062 sec / 0.000 sec
121623	11:11:12	SELECT * FROM credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.094 sec / 0.000 sec
121624	11:12:07	SELECT * FROM credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.000 sec / 0.000 sec
121625	11:16:05	SELECT * FROM credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.000 sec / 0.000 sec

Object Info Session

Query Completed

Procedimiento

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

- company_name
- phone
- email
- country
- website
- Indexes
- Foreign Keys
- Triggers
- credit_card
 - Columns
 - id
 - iban
 - pan
 - pin
 - cvv

Administration Schemas

Information

Schema: transactions

SQL File 18" x SQL File 19"

57 WHERE id NOT IN (
58 SELECT DISTINCT company_id
59 FROM transaction
60 }
61 -- no hay empresas
62
63 #Ejercicio 5
64 -- cambio en el base de datos del numero iban para una tarjeta de credito
65 • SELECT *
66 FROM credit_card
67 WHERE id = "CcU-2938";
68
69 -- modificar
70 • UPDATE credit_card
71 SET iban = 'TR323456312213576817699999'
72 WHERE id = 'CcU-2938';
73

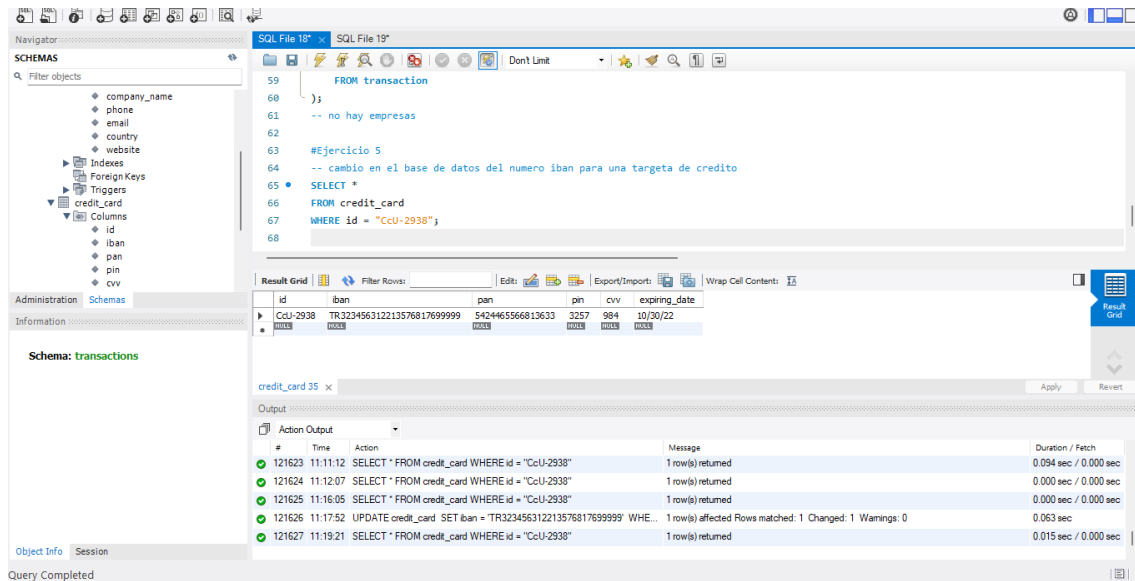
Output

#	Time	Action	Message	Duration / Fetch
121622	11:00:46	SELECT * FROM credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.062 sec / 0.000 sec
121623	11:11:12	SELECT * FROM credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.094 sec / 0.000 sec
121624	11:12:07	SELECT * FROM credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.000 sec / 0.000 sec
121625	11:16:05	SELECT * FROM credit_card WHERE id = "CcU-2938"	1 row(s) returned	0.000 sec / 0.000 sec
121626	11:17:52	UPDATE credit_card SET iban = 'TR323456312213576817699999' WHE...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.063 sec

Object Info Session

Query Completed

consulta después del cambio del IBAN



Ejercicio 6

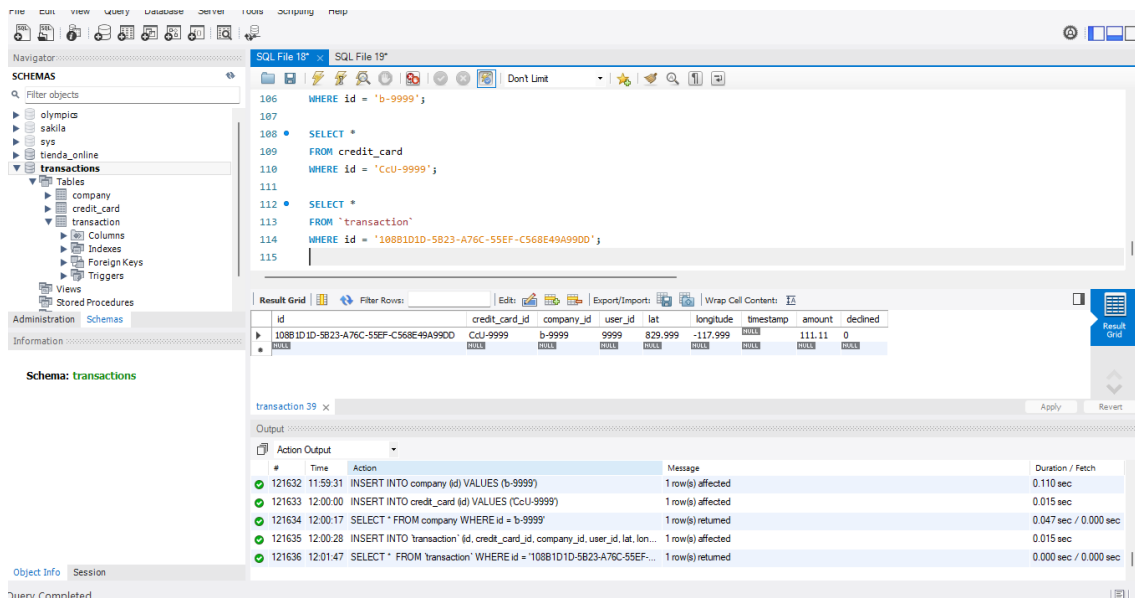
En la tabla "transaction" ingresa una nueva transacción con la siguiente información:

```

Id 108B1D1D-5B23-A76C-55EF-C568E49A99DD credit_card_id CcU-9999 company_id b-9999
user_id 9999 lat 829.999 longitude -117.999 amount 111.11 declined 0

```

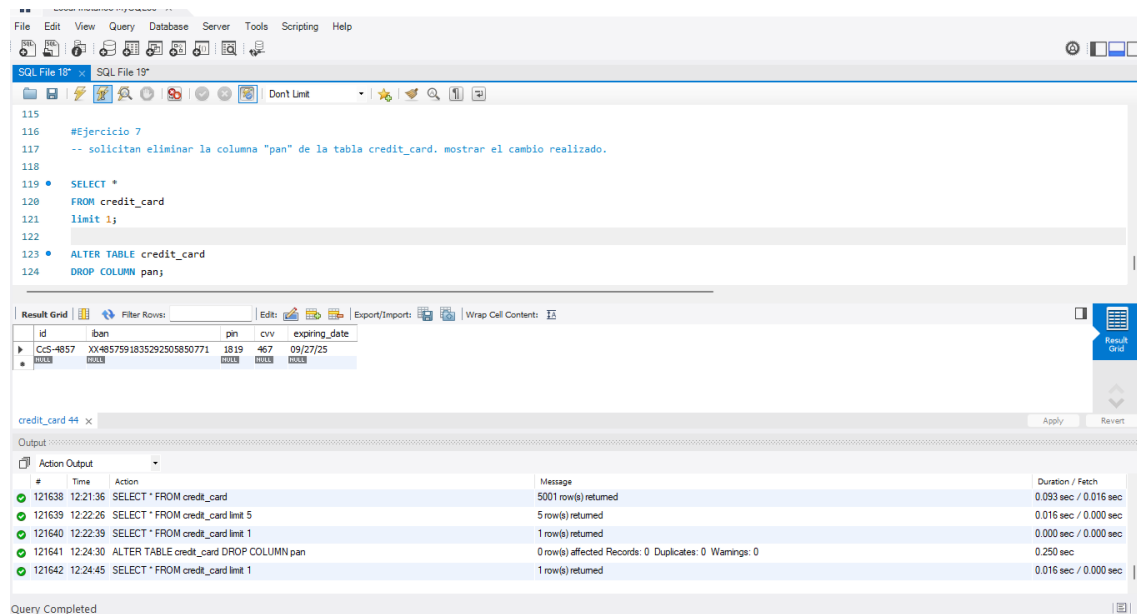
RESPUESTA: para este ejercicio se debe también crear el campo Id en las tablas company y credit_card, por ser Foreign Key en la tabla transaction dependen de su Primary Key en sus tablas correspondientes.



Ejercicio 7

Desde recursos humanos te solicitan eliminar la columna "pan" de la tabla credit_card. Recuerda mostrar el cambio realizado.

Se puede observar la tabla de credit_card sin la columna pan



Ejercicio 8

Descarga los archivos CSV que encontrarás en el apartado de recursos:

american_users.csv

european_users.csv

companies.csv

credit_cards.csv

transactions.csv

Estudia y diseña una base de datos con un esquema de estrella que contenga, al menos 4 tablas de las que puedas realizar las siguientes consultas:
La tabla de products.csv la utilizaremos más adelante.

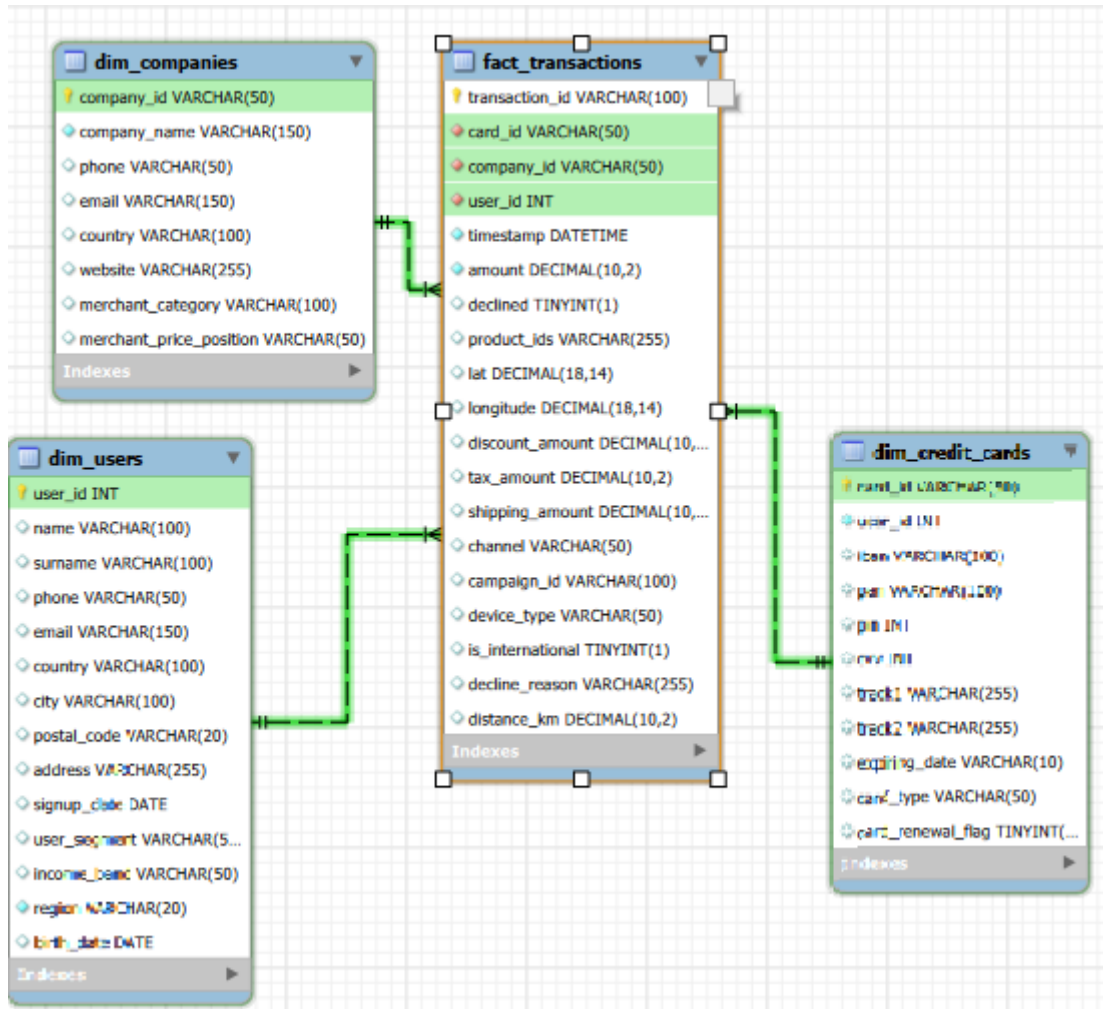
Diseño, Implementación y Carga del Modelo de Datos Analytics

1. Diseño del Modelo de Datos (Esquema de Estrella)

Se diseñó e implementó un modelo relacional bajo un Esquema de Estrella (Star Schema) optimizado para consultas analíticas dentro del esquema e_commerce_analytics. El diseño está compuesto por cuatro tablas principales:

- Tabla de Hechos (fact_transactions): Centraliza las métricas del negocio (amount, discount_amount, tax_amount, shipping_amount), estados de las operaciones y las coordenadas geográficas.

- Tablas de Dimensiones:
 - dim_users: Dimensión consolidada y unificada de clientes.
 - dim_companies: Información de comercios y perfiles de mercado.
 - dim_credit_cards: Información analítica sobre los métodos de pago.



2. Estrategia Segura de Carga de Datos (ETL)

Para cumplir con las directivas estrictas de seguridad nativas de MySQL 8.0+ (--secure-file-priv), evitar brechas de acceso y no comprometer el servidor modificando archivos internos del sistema operativo, se implementó el siguiente flujo automatizado:

1. Aislamiento en Carpeta Segura: Se utilizó el Símbolo del Sistema (CMD) en modo Administrador para copiar de manera forzada la totalidad de los archivos CSV hacia el directorio privado y protegido del servidor (C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\).

```
FacturaTech - Tecnología en facturación a tu alcance | Login
https://plataforma.facturatech.co/login/

C:\Users\57322> 1. Copiar Usuarios Americanos
C:\Users\57322>copy "C:\Users\57322\Documents\Especializacion Analista de Datos\N1-Ex.8__american_users.csv" "C:\Program
Data\MySQL\MySQL Server 8.0\Uploads\"
1 archivo(s) copiado(s).

C:\Users\57322>
C:\Users\57322> 2. Copiar Usuarios Europeos
C:\Users\57322>copy "C:\Users\57322\Documents\Especializacion Analista de Datos\N1-Ex.8__european_users.csv" "C:\Program
Data\MySQL\MySQL Server 8.0\Uploads\"
1 archivo(s) copiado(s).

C:\Users\57322>
C:\Users\57322> 3. Copiar Compañías
C:\Users\57322>copy "C:\Users\57322\Documents\Especializacion Analista de Datos\N1-Ex.8__companies.csv" "C:\ProgramData\
MySQL\MySQL Server 8.0\Uploads\"
1 archivo(s) copiado(s).

C:\Users\57322>
C:\Users\57322> 4. Copiar Tarjetas de Crédito
C:\Users\57322>copy "C:\Users\57322\Documents\Especializacion Analista de Datos\N1-Ex.8__credit_cards.csv" "C:\ProgramDa
ta\MySQL\MySQL Server 8.0\Uploads\"
1 archivo(s) copiado(s).

C:\Users\57322>
C:\Users\57322> 5. Copiar Transacciones
C:\Users\57322>copy "C:\Users\57322\Documents\Especializacion Analista de Datos\N1-Ex.8__transactions.csv" "C:\ProgramDa
ta\MySQL\MySQL Server 8.0\Uploads\"
1 archivo(s) copiado(s).
```

2. Ejecución Nativa: Una vez aislados, los datos se importaron mediante sentencias LOAD DATA INFILE directas desde MySQL Workbench sin requerir la cláusula LOCAL, garantizando un proceso veloz y alineado con los estándares de seguridad corporativos.

3. Tratamiento, Unificación y Transformación de Datos

Durante la fase de carga e ingesta mediante código SQL, se aplicaron reglas de transformación de datos para garantizar la calidad de la información:

- Identificación de Delimitadores: Se identificó que la fuente transactions.csv poseía una estructura con delimitador por punto y coma (;), configurándose adecuadamente en el script de carga (FIELDS TERMINATED BY ';'), mientras que el resto de las dimensiones se procesaron con comas (,).
- Unificación Usuarios y Trazabilidad (dim_users): Los archivos independientes american_users.csv y european_users.csv se unificaron de forma consecutiva dentro de la misma dimensión de destino. Para mantener la trazabilidad de origen sin que existiera en el CSV original, se inyectó dinámicamente un atributo calculado (SET region = 'America' / 'Europe').
- Estandarización de Fechas: Las fechas de nacimiento originales venían guardadas en formato de texto regional (ej. "Jul 2, 1980"). Se ejecutó un proceso de limpieza posterior utilizando funciones de conversión (STR_TO_DATE) y limpieza de caracteres (REPLACE) para transformarlas al tipo de dato nativo DATE (YYYY-MM-DD).

```
Sprint2_eje1-7_nivel1 Sprint2_eje8-10_nivel1* SQL File 23*
85 -- Carga de American Users
86 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__american_users.csv'
87 INTO TABLE dim_users
88 FIELDS TERMINATED BY ','
89 ENCLOSED BY '"'
90 LINES TERMINATED BY '\n'
91 IGNORE 1 LINES
92 (user_id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band)
93 SET region = 'America'; -- Asigna directamente el valor a la columna region
94
95 -- Carga de European Users (Se unifican automáticamente en la misma tabla)
96 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__european_users.csv'
97 INTO TABLE dim_users
98 FIELDS TERMINATED BY ','
99 ENCLOSED BY '"'
100 LINES TERMINATED BY '\n'
101 IGNORE 1 LINES
102 (user_id, name, surname, phone, email, birth_date, country, city, postal_code, address, signup_date, user_segment, income_band)
103 SET region = 'Europe'; -- Asigna directamente el valor a la columna region
104
105 -- Estandarización y limpieza automática de las fechas de nacimiento
106 • ALTER TABLE dim_users ADD COLUMN birth_date_clean DATE;
107
108 • UPDATE dim_users
109 SET birth_date_clean = STR_TO_DATE(REPLACE(birth_date, '-', ''), '%b %e, %Y')
110 WHERE birth_date IS NOT NULL AND birth_date != '';
111
112 • ALTER TABLE dim_users DROP COLUMN birth_date;
113
```

```
117 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__companies.csv'
118 INTO TABLE dim_companies
119 FIELDS TERMINATED BY ','
120 ENCLOSED BY '"'
121 LINES TERMINATED BY '\n'
122 IGNORE 1 LINES
123 (company_id, company_name, phone, email, country, website, merchant_category, merchant_price_position);
124
125 -- Carga de Tarjetas de Crédito
126 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__credit_cards.csv'
127 INTO TABLE dim_credit_cards
128 FIELDS TERMINATED BY ','
129 ENCLOSED BY '"'
130 LINES TERMINATED BY '\n'
131 IGNORE 1 LINES
132 (card_id, user_id, iban, pan, pin, cvv, track1, track2, expiring_date, card_type, card_renewal_flag);
133
134 -- CARGA DE LA TABLA DE HECHOS (Transactions)
135 -- Este archivo usa punto y coma (;) como delimitador
136 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__transactions.csv'
137 INTO TABLE fact_transactions
138 FIELDS TERMINATED BY '; '
139 ENCLOSED BY '"'
140 LINES TERMINATED BY '\n'
141 IGNORE 1 LINES
142 (transaction_id, card_id, company_id, timestamp, amount, declined, product_ids, user_id, lat, longitude, discount_amount, tax_amount, shipping_amount, channel, campaign_id, device_t
143
144 • SET SQL_SAFE_UPDATES = 1;
```

Ejercicio 9

Realiza una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'e_commerce_analytics' schema with tables like dim_users, fact_transactions, and others. The central pane shows a SQL query:

```
147 #Ejercicio 9
148 -- Consulta que extrae la información descriptiva del usuario
149 SELECT u.user_id, u.name, u.surname, u.email, u.country, u.region
150 FROM dim_users u
151 WHERE u.user_id IN (SELECT t.user_id
152                     FROM fact_transactions t
153                     GROUP BY t.user_id
154                     HAVING COUNT(t.transaction_id) > 80
155                    )
156 ORDER BY u.region, u.surname;
```

The right pane shows the 'Result Grid' with 6 columns: user_id, name, surname, email, country, and region. It displays 5 rows of data for users from Italy, United Kingdom, Germany, and Poland.

Below the result grid, the 'Output' pane shows the execution plan with 6 rows of action output, including the query execution time and duration.

Ejercicio 10

Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd., utiliza por lo menos 2 tablas.

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'e_commerce_analytics' schema. The central pane shows a SQL query:

```
160 SELECT c.iban AS iban_tarjeta, c.card_type AS tipo_tarjeta, ROUND(AVG(t.amount), 2) AS promedio_transaccion
161 FROM fact_transactions t
162 JOIN dim_credit_cards c ON t.card_id = c.card_id
163 WHERE c.card_type IN ('credit', 'premium_credit')
164 AND t.company_id = (SELECT company_id
165                   FROM dim_companies
166                   WHERE company_name = 'Donec Ltd')
167
168 GROUP BY c.iban, c.card_type
169 ORDER BY promedio_transaccion DESC;
```

The right pane shows the 'Result Grid' with 3 columns: iban_tarjeta, tipo_tarjeta, and promedio_transaccion. It displays 6 rows of data for credit and premium credit cards.

Below the result grid, the 'Output' pane shows the execution plan with 6 rows of action output, including the query execution time and duration.

Nivel 2

Ejercicio 1

Identifica los cinco días que se generó la mayor cantidad de ingresos en la empresa por ventas. Muestra la fecha de cada transacción junto con el total de las ventas.

Ejercicio 1

```

-- Mostrar cinco días con mayor cantidad de ingresos en la empresa por ventas. Muestra fecha con total de las ventas
SELECT DATE(t.timestamp) AS fecha_transaccion, ROUND(SUM(t.amount), 2) AS total_ventas_dia
FROM fact_transactions t
WHERE t.declined = 0
GROUP BY DATE(t.timestamp)
ORDER BY total_ventas_dia DESC
LIMIT 5;

```

fecha_transaccion	total_ventas_dia
2023-02-20	15728.40
2019-11-18	15447.89
2022-12-13	14944.00
2019-03-18	14896.47
2017-12-20	14367.96

Ejercicio 2

Presenta el nombre, teléfono, país, fecha y amount, de aquellas empresas que realizaron transacciones con un valor comprendido entre 350 y 400 euros y en alguna de estas fechas: 29 de abril de 2015, 20 de julio de 2018 y 13 de marzo de 2024. Ordena los resultados de mayor a menor cantidad.

Ejercicio 2

```

-- nombre, teléfono, país, fecha y amount, de transacciones entre 350 y 400 euros,
-- en las fechas 9/04/2015, 20/07/2018 y 13/03/2024, ordenar de mayor a menor
SELECT c.company_name AS nombre_empresa, c.phone AS telefono, c.country AS país, DATE(t.timestamp) AS fecha_transaccion, t.amount AS monto_euro
FROM fact_transactions t
JOIN dim_companies c ON t.company_id = c.company_id
WHERE t.amount BETWEEN 350 AND 400 -- Filtro 1: Rango de valores monetarios solicitado
AND DATE(t.timestamp) IN ('2015-04-29', '2018-07-20', '2024-03-13') -- Filtro 2: Restricción a las fechas específicas requeridas
AND t.declined = 0 -- Filtro solo transacciones aprobadas
ORDER BY t.amount DESC;

```

nombre_empresa	telefono	país	fecha_transaccion	monto_euro
Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29	390.33
Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13	383.58
Mauris Institute	+75 473 619 9472	Sweden	2015-04-29	369.20
Pede Cum Ltd	+26 625 798 6126	Norway	2018-07-20	358.64
Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands	2024-03-13	352.21
Tempor Diam Institute	+89 848 141 3136	Netherlands	2024-03-13	350.71

Ejercicio 3

Necesitamos optimizar la asignación de los recursos y dependerá de la capacidad operativa que se requiera, por lo que te piden la información sobre la cantidad de transacciones que realizan las empresas, pero el departamento de recursos humanos es exigente y quiere un listado de las empresas en las que especifiques si tienen igual o más de 400 transacciones o menos.

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

e_commerce_analytics

Tables

dim_companies

dim_credit_cards

dim_users

fact_transactions

Views

Stored Procedures

Functions

olympics

sakila

sys

tienda_online

transactions

Administration Schemas

Information

Schema: e_commerce_analytics

SQL File 23*

```

22 #Ejercicio 3
23 -- cantidad de transacciones que realizan las empresas, si tienen igual o más de 400 transacciones o menos.
24 • SELECT c.company_name AS nombre_empresa, c.country AS país, COUNT(t.transaction_id) AS total_transacciones,
25     CASE
26     WHEN COUNT(t.transaction_id) >= 400 THEN '+= 400 transacciones'
27     ELSE '- 400 transacciones'
28     END AS clasificacion
29 FROM dim_companies c
30 LEFT JOIN fact_transactions t ON c.company_id = t.company_id
31 GROUP BY c.company_id, c.company_name, c.country
32 ORDER BY total_transacciones DESC;

```

Result Grid

nombre_empresa	país	total_transacciones	clasificacion
Ac Fermentum Incorporated	Germany	2401	+= 400 transacciones
Nunc Interdum Incorporated	Germany	1599	+= 400 transacciones
Donec Fringilla PC	France	1593	+= 400 transacciones
Mauris Institute	Sweden	1586	+= 400 transacciones
Rutrum Non Inc.	Germany	1585	+= 400 transacciones
Aliquet Vel Vulputate Incorporated	Netherlands	1583	+= 400 transacciones
Aliquam PC	Germany	1576	+= 400 transacciones
Orci Adipiscing Limited	United Kingdom	1567	+= 400 transacciones
Etiam Blandum Fermentum Industries	France	1566	+= 400 transacciones

Result 7 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
8	10:53:44	SELECT c.company_name AS nombre_empresa, c.country AS país, COUNT(t.transaction_id) AS total_transacciones, CASE WHEN COUNT(t.transaction_id) >= 400 THEN '+= 400 transacciones' ELSE '- 400 transacciones' END AS clasificacion FROM dim_companies c LEFT JOIN fact_transactions t ON c.company_id = t.company_id GROUP BY c.company_id, c.company_name, c.country ORDER BY total_transacciones DESC;	100 row(s) returned	0.375 sec / 0.000 sec
9	10:55:27	SELECT c.company_name AS nombre_empresa, c.country AS país, COUNT(t.transaction_id) AS total_transacciones, CASE WHEN COUNT(t.transaction_id) >= 400 THEN '+= 400 transacciones' ELSE '- 400 transacciones' END AS clasificacion FROM dim_companies c LEFT JOIN fact_transactions t ON c.company_id = t.company_id GROUP BY c.company_id, c.company_name, c.country ORDER BY total_transacciones DESC;	100 row(s) returned	0.375 sec / 0.000 sec

Object Info Session

Ejercicio 4

Elimina de la tabla transacción el registro con ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.

Consulta antes del borrado

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHMAS

Filter objects

e_commerce_analytics

Tables

dim_companies

dim_credit_cards

dim_users

fact_transactions

Views

Stored Procedures

Functions

olympics

sakila

sys

tienda_online

transactions

Administration Schemas

Information

Schema: e_commerce_analytics

SQL File 23*

```

34 #Ejercicio 4
35 -- Elimina de la tabla transacción el registro con ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.
36 • SELECT transaction_id, amount, timestamp
37 FROM fact_transactions
38 WHERE transaction_id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
39
40
41 DELETE FROM fact_transactions
42 WHERE transaction_id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
43
44

```

Result Grid

transaction_id	amount	timestamp
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	284.14	2021-12-16 02:40:48

fact_transactions 10 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
25	11:12:02	SELECT * FROM fact_transactions WHERE transaction_id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';	1 row(s) returned	0.000 sec / 0.000 sec
26	11:12:16	DELETE FROM fact_transactions WHERE transaction_id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';	1 row(s) returned	0.015 sec / 0.000 sec

Object Info Session

Ejecución de la eliminación:

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'e_commerce_analytics' schema with tables like dim_companies, dim_credit_cards, dim_users, and fact_transactions. The main editor shows two SQL queries:

```

36 SELECT transaction_id, amount, timestamp
37 FROM fact_transactions
38 WHERE transaction_id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
39
40
41 DELETE FROM fact_transactions
42 WHERE transaction_id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
43

```

The 'Output' tab shows the execution results for these queries. The first query (SELECT) returned 1 row. The second query (DELETE) also returned 1 row. The 'Action Output' tab shows the execution details for the DELETE query, including the message 'Error Code: 1054. Unknown column 'expiration_date' in field list'.

Consulta después de la eliminación:

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'e_commerce_analytics' schema. The main editor shows a SQL query:

```

42 WHERE transaction_id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
43
44
45 SELECT transaction_id
46 FROM fact_transactions
47 WHERE transaction_id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';

```

The 'Output' tab shows the execution results for the SELECT query. The 'Result Grid' shows the results of the query, which returned 1 row. The 'Action Output' tab shows the execution details for the SELECT query, including the message 'Error Code: 1054. Unknown column 'expiration_date' in field list'.

Ejercicio 5

La sección de marketing desea tener acceso a información específica para realizar análisis y estrategias efectivas. Se ha solicitado crear una vista que proporcione detalles clave sobre las compañías y sus transacciones. Será necesaria que crees una vista llamada VistaMarketing que contenga la siguiente información: Nombre de la compañía. Teléfono de contacto. País de residencia. Media de compra realizado por cada compañía. Presenta la vista creada, ordenando los datos de mayor a menor promedio de compra.

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'e_commerce_analytics' schema with various tables and views. The central pane shows a SQL query file with the following code:

```

45 SELECT transaction_id
46 FROM fact_transactions
47 WHERE transaction_id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
48
49 #Ejercicio 5
50 -- VistaMarketing, tabla virtual con Nombre de la compañía, teléfono de contacto, país de residencia, Media de compra.
51 -- ordenar de mayor a menor
52 CREATE OR REPLACE VIEW VistaMarketing AS
53 SELECT c.company_name AS Nombre_Compañía, c.phone AS Telefono_Contacto, c.country AS Pais_Residencia, ROUND(AVG(t.amount), 2) AS Promedio_Compra
54 FROM dim_companies c
55 JOIN fact_transactions t ON c.company_id = t.company_id
56 WHERE t.declined = 0 OR t.declined IS NULL
57 GROUP BY c.company_id, c.company_name, c.phone, c.country;

```

The right pane shows the 'Output' window with a table of execution results:

#	Time	Action	Message	Duration / Fetch
29	11:27:24	SELECT DISTINCT company_name FROM company WHERE id IN (SELECT compan...	Error Code: 1146. Table 'e_commerce_analytics.company' doesn't exist	0.032 sec
30	11:27:53	SELECT DISTINCT company_name FROM company WHERE id IN (SELECT compan...	100 row(s) returned	0.546 sec / 0.000 sec
31	11:30:34	SELECT company_name FROM company WHERE id NOT IN (SELECT DISTINCT...	0 row(s) returned	0.234 sec / 0.000 sec
32	11:56:28	CREATE OR REPLACE VIEW VistaMarketing AS SELECT c.company_name AS N...	0 row(s) affected	0.125 sec
33	11:56:53	SELECT * FROM VistaMarketing ORDER BY Promedio_Compra DESC	101 row(s) returned	1.156 sec / 0.000 sec
34	12:05:38	CREATE OR REPLACE VIEW VistaMarketing AS SELECT c.company_name AS Nom...	0 row(s) affected	0.047 sec
35	12:09:17	CREATE OR REPLACE VIEW VistaMarketing AS SELECT c.company_name AS Nom...	0 row(s) affected	0.016 sec
36	12:10:05	SELECT * FROM VistaMarketing ORDER BY Promedio_Compra DESC	100 row(s) returned	0.641 sec / 0.000 sec
37	12:10:26	CREATE OR REPLACE VIEW VistaMarketing AS SELECT c.company_name AS Nom...	0 row(s) affected	0.000 sec

Tabla creada:

The screenshot shows the SQL Server Enterprise Manager interface. The left pane displays the 'e_commerce_analytics' schema. The central pane shows a SQL query file with the following code:

```

56 WHERE t.declined = 0 OR t.declined IS NULL
57 GROUP BY c.company_id, c.company_name, c.phone, c.country;
58
59 SELECT *
60 FROM VistaMarketing
61 ORDER BY Promedio_Compra DESC;

```

The right pane shows the 'Result Grid' window with a table of execution results:

Nombre_Compañía	Telefono_Contacto	Pais_Residencia	Promedio_Compra
Ac Fermentum Incorporated	+64 601 244 6320	Germany	330.71
Ut Semper Foundation	+10 283 500 6449	Sweden	307.30
Dolor Vitae Limited	+6 421 567 9195	France	306.21
Orci Adipiscing Limited	+60 897 390 5820	United Kingdom	305.87
Nunc In Foundation	+13 267 741 2619	Italy	305.67

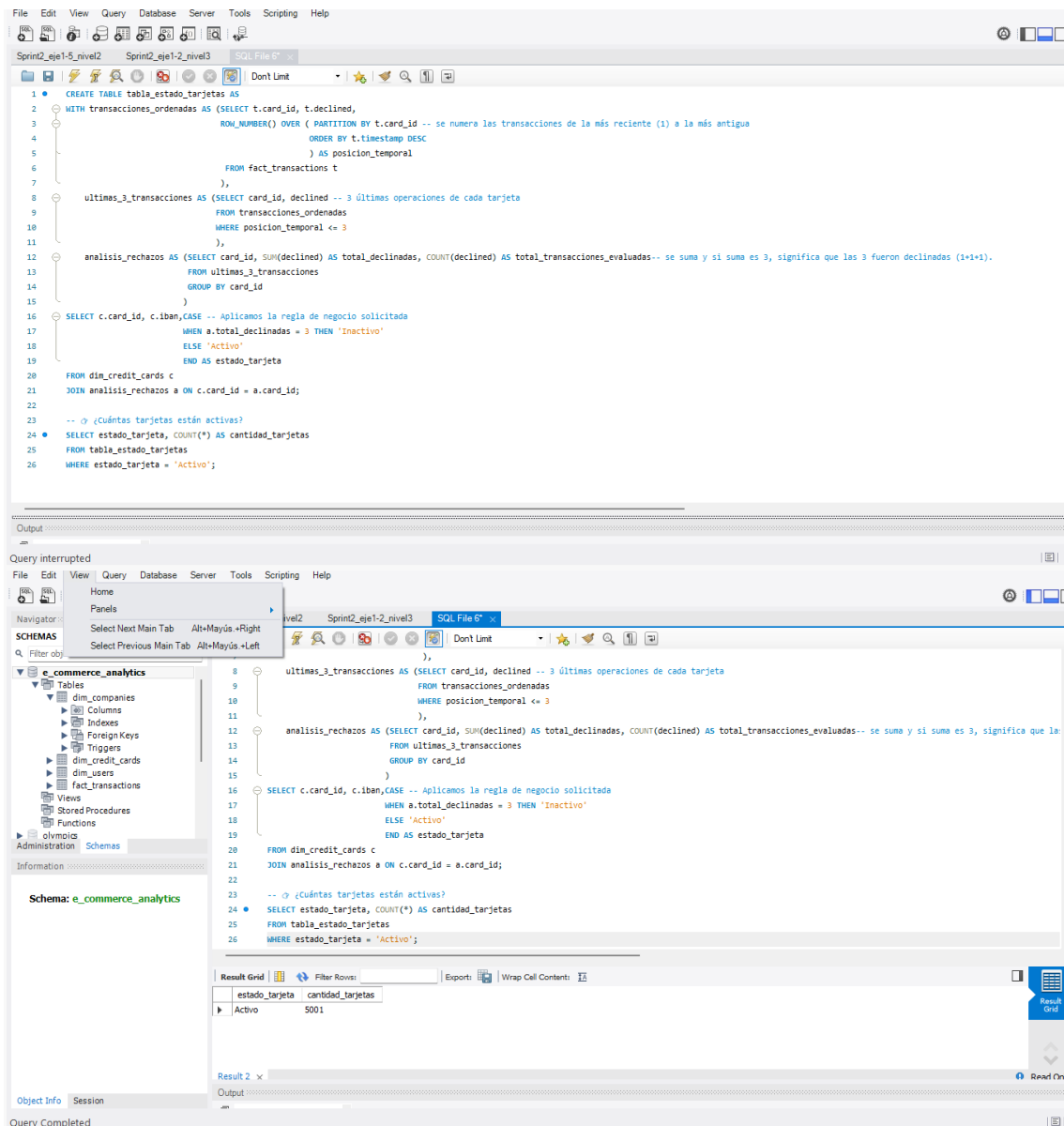
Nivel 3

Ejercicio 1

Crea una nueva tabla que refleje el estado de las tarjetas de crédito basado en si las tres últimas transacciones han sido declinadas entonces es inactivo, si al menos una no es rechazada entonces es activo. Partiendo de esta tabla responde:

¿Cuántas tarjetas están activas?

Respuesta: 5001 tarjetas estan activas



Ejercicio 2

Crea una tabla con la que podamos unir los datos del archivo de products.csv con la base de datos creada (ya que hasta ahora no podíamos hacerlo), teniendo en cuenta que desde transaction tienes product_ids. Genera la siguiente consulta:

Necesitamos conocer el número de veces que se ha vendido cada producto.

Respuesta:

Detectamos que en la tabla de transacciones fact_transactions, los identificadores de los productos vendidos estaban guardados en una sola celda como texto separado por comas (por ejemplo: '9, 11, 47, 50'). Esto hacía imposible cruzar la información

directamente con el catálogo de productos `products.csv` usando un JOIN tradicional.

Para solucionar este desajuste de diseño, creamos una nueva tabla pasarela llamada `transaction_products`.

Usamos la herramienta avanzada `JSON_TABLE` para "desenrollar" las cadenas de texto con comas y convertirlas en un formato de lista real.

Con esto, logramos que, si una transacción contenía 4 productos, el sistema creara de forma automática 4 filas independientes asociando limpiamente cada `transaction_id` con un único `product_id`.

Preparamos la tabla maestra `dim_products` para albergar los datos del archivo `.csv`.

Solucionamos las restricciones del "Modo de Actualización Segura" (`SQL_SAFE_UPDATES = 0`) para limpiar los símbolos de dólar (\$) que venían arrastrándose en las columnas de precios y costos.

Modificamos la estructura para que los tipos de datos fueran numéricos reales (`DECIMAL`) y adaptamos las consultas para que buscaran el identificador utilizando la columna exacta del archivo, que se llamaba simplemente `id`.

Con ambas tablas listas, normalizadas y pobladas, ejecutamos un `LEFT JOIN` uniendo el `id` de la tabla de productos con el `product_id` de la tabla pasarela.

Usamos la función `COUNT(tp.product_id)` agrupando por cada artículo para obtener la métrica exacta de ventas.

Ordenamos los resultados de mayor a menor (`ORDER BY ... DESC`) para situar en la cima del reporte los productos más vendidos (Best Sellers) y conservar al final (con valor 0) aquellos que no han registrado salida en el histórico comercial.

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

e_commerce_analytics

Tables

- dim_companies
- dim_credit_cards
- dim_users
- fact_transactions
- tabla_estado_tarjetas
- transaction_products

Views

Stored Procedures

Functions

olympics

sakila

svs

Administration Schemas

Information

Schema: e_commerce_analytics

Object Info Session

Query Completed

SQL File 6*

Don't Limit

```

25
26 -- ¿Cuántas tarjetas están activas?
27 • SELECT estado_tarjeta, COUNT(*) AS cantidad_tarjetas
28 FROM tabla_estado_tarjetas
29 WHERE estado_tarjeta = 'Activo';
30
31 #Ejercicio 2
32 -- crear tabla para unir datos del archivo de products.csv con la base de datos creada
33 • CREATE TABLE transaction_products (
34     transaction_id VARCHAR(50),
35     product_id INT,
36     PRIMARY KEY (transaction_id, product_id)
37 );

```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
34	12:05:38	CREATE OR REPLACE VIEW VistaMarketing AS SELECT c.company_name AS Nom...	0 row(s) affected	0.047 sec
35	12:09:17	CREATE OR REPLACE VIEW VistaMarketing AS SELECT c.company_name AS Nom...	0 row(s) affected	0.016 sec
36	12:10:05	SELECT * FROM VistaMarketing ORDER BY Promedio_Compra DESC	100 row(s) returned	0.641 sec / 0.000 sec
37	12:10:26	CREATE OR REPLACE VIEW VistaMarketing AS SELECT c.company_name AS Nom...	0 row(s) affected	0.000 sec
38	12:12:30	SELECT * FROM VistaMarketing ORDER BY Promedio_Compra DESC	100 row(s) returned	0.469 sec / 0.000 sec
39	12:33:53	CREATE TABLE tabla_estado_tarjetas AS WITH transacciones_ordenadas AS (SE...	5001 row(s) affected Records: 5001 Duplicates: 0 Warnings: 0	1.047 sec
40	12:34:09	SELECT estado_tarjeta, COUNT(*) AS cantidad_tarjetas FROM tabla_estado_...	1 row(s) returned	0.016 sec / 0.000 sec
41	13:28:39	CREATE TABLE tabla_estado_tarjetas AS WITH transacciones_ordenadas AS (SELE...	Error Code: 1050. Table 'tabla_estado_tarjetas' already exists	0.172 sec
42	13:30:17	SELECT estado_tarjeta, COUNT(*) AS cantidad_tarjetas FROM tabla_estado_tarjetas ...	1 row(s) returned	0.063 sec / 0.000 sec
43	13:59:09	CREATE TABLE transaction_products (transaction_id VARCHAR(50), product_id...	0 row(s) affected	0.250 sec

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

e_commerce_analytics

Tables

- dim_companies
- dim_credit_cards
- dim_products
- dim_users
- fact_transactions
- tabla_estado_tarjetas
- transaction_products

Views

Stored Procedures

Functions

olympics

Administration Schemas

Information

Table: dim_products

Columns:

- id int PK
- product_name varchar(150)
- price varchar(50)
- colour varchar(30)
- weight decimal(10,2)
- warehouse_id varchar(50)
- category varchar(100)
- brand varchar(100)
- cost varchar(50)
- launch_date date

Object Info Session

SQL File 1*

Don't Limit

```

35
36 );
37 • INSERT INTO transaction_products (transaction_id, product_id)
38 SELECT t.transaction_id, CAST(j.product_id AS UNSIGNED) AS product_id
39 FROM fact_transactions t
40 JOIN
41 -- Transformamos la cadena '9, 11, 47' en un formato JSON '[9, 11, 47]'
42 JSON_TABLE( CONCAT('[' , t.product_ids , ']', '[*]' ) COLUMNS (product_id VARCHAR(50) PATH '$')
43 ) AS j
44 -- solo procesar si hay productos y la transacción fue aprobada
45 WHERE t.product_ids IS NOT NULL
46 AND t.product_ids <> ''
47 AND t.declined = 0;
48

```

Result Grid

id_producto	nombre_producto	precio_euro	categoria	cantidad_veces_vendido
52	riverlands the duel	31.52	Food	2635
29	Tully maester Taryl	167.20	Books	2607
16	the duel warden	180.91	Home	2583
48	rock Renly in	21.53	Beauty	2580
21	duel Drowlf	96.90	Books	2579
87	sith Jade	96.26	Food	2575

Result 2 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
39	19:13:51	SELECT p.id AS id_producto, p.product_name AS nombre_producto, p.price AS ...	100 row(s) returned	1.922 sec / 0.000 sec
40	19:24:15	SELECT p.id AS id_producto, p.product_name AS nombre_producto, p.price AS precio...	100 row(s) returned	0.719 sec / 0.000 sec

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

e_commerce_analytics

Tables

- dim_companies
- dim_credit_cards
- dim_products
- dim_users
- fact_transactions
- tabla_estado_tarjetas
- transaction_products

Views

Stored Procedures

Functions

olympics

Administration Schemas

Information

Table: dim_products

Columns:

- id int PK
- product_name varchar(150)
- price varchar(50)
- colour varchar(30)
- weight decimal(10,2)
- warehouse_id varchar(50)
- category varchar(100)
- brand varchar(100)
- cost varchar(50)
- launch_date date

Object Info Session

SQL File 1*

Don't Limit

```

41 -- Transformamos la cadena '9, 11, 47' en un formato JSON '[9, 11, 47]'
42 JSON_TABLE( CONCAT('[' , t.product_ids , ']', '[*]' ) COLUMNS (product_id VARCHAR(50) PATH '$')
43 ) AS j
44 -- solo procesar si hay productos y la transacción fue aprobada
45 WHERE t.product_ids IS NOT NULL
46 AND t.product_ids <> ''
47 AND t.declined = 0;
48
49 -- número de veces que se ha vendido cada producto
50 • SELECT p.id AS id_producto, p.product_name AS nombre_producto, p.price AS precio_euro, p.category AS categoria, COUNT(tp.product_id) AS cantidad_veces_vendido
51 FROM dim_products p
52 JOIN transaction_products tp ON p.id = tp.product_id
53 GROUP BY p.id, p.product_name, p.price, p.category
54 ORDER BY cantidad_veces_vendido DESC;

```

Result Grid

id_producto	nombre_producto	precio_euro	categoria	cantidad_veces_vendido
52	riverlands the duel	31.52	Food	2635
29	Tully maester Taryl	167.20	Books	2607
16	the duel warden	180.91	Home	2583
48	rock Renly in	21.53	Beauty	2580
21	duel Drowlf	96.90	Books	2579
87	sith Jade	96.26	Food	2575

Result 2 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
39	19:13:51	SELECT p.id AS id_producto, p.product_name AS nombre_producto, p.price AS ...	100 row(s) returned	1.922 sec / 0.000 sec
40	19:24:15	SELECT p.id AS id_producto, p.product_name AS nombre_producto, p.price AS precio...	100 row(s) returned	0.719 sec / 0.000 sec

SQL script saved to 'C:\Users\57322\Documents\Especializacion Analista de Datos\Sprint2_eje1-2_nivel3.sql'